

musiccloud Resolver Flow

musiccloud

Version 1.3.0+14e3c2f-dirty, 2026-06-06

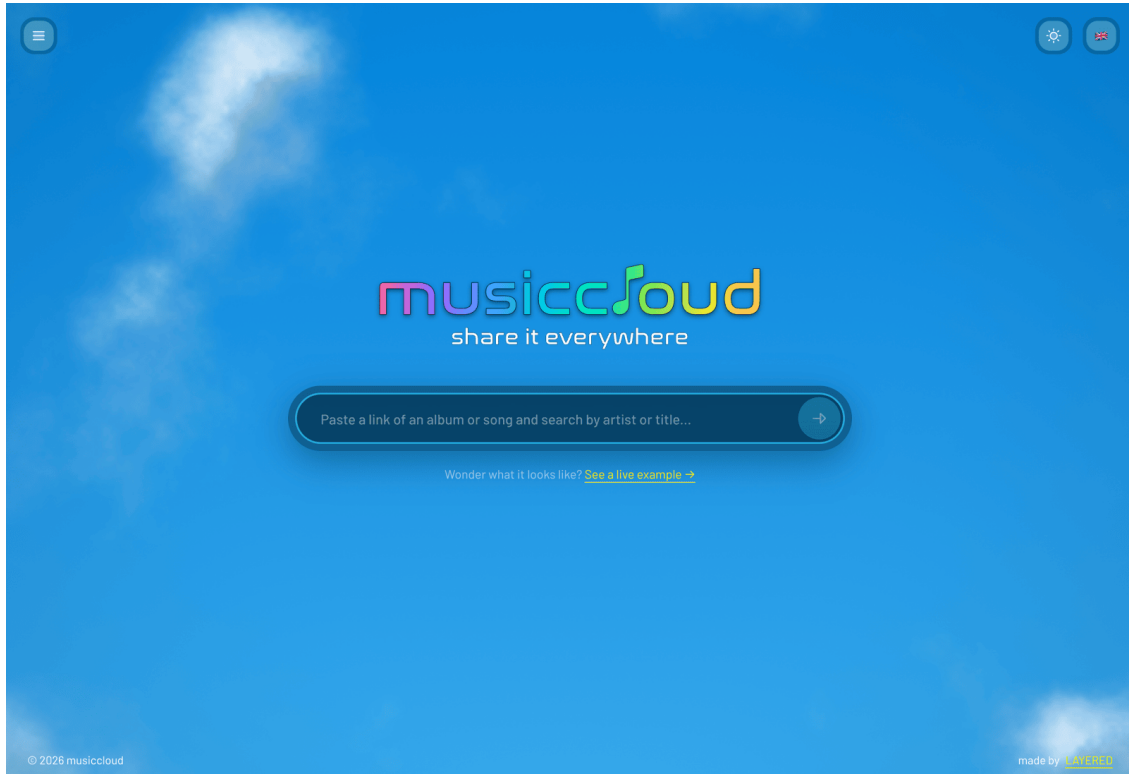
Contents

1 Overview	2
2 Endpoints	2
3 Query Shapes	3
3.1 Streaming-Service URL	3
3.2 Free-Text Query	4
3.3 Genre-Discovery Query	5
4 Structured Search	8
4.1 Purpose	8
4.2 Syntax	8
4.3 Validation	8
4.4 Response	8
5 selectedCandidate Follow-Up	9
6 Error Cases	9
7 Practical Examples	9

1 Overview

The resolver accepts a link or search query and turns it into a unified musiccloud response. The response is either a resolved track, album, or artist, a candidate list for an ambiguous track search, a genre-discovery result, or an error with a stable error code.

In the musiccloud web app, every query starts at a single input. The screenshot below shows that entry point: it accepts a streaming-service URL, free text, a structured `title:/artist:` query, or a `genre: discovery` query.



The single input that accepts every query shape.

This document describes the public query shapes from a user's perspective. Implementation details such as enabled plugins, admin toggles, or internal cache tables are not required to use the API. The screenshots illustrate how each query shape appears in the web app; the JSON blocks show the matching API request and response.

2 Endpoints

Endpoint	Use
POST /api/v1/resolve	Full resolve endpoint for API clients with an X-API-Key or bearer token. Supports URLs, free text, genre discovery, structured search, and the <code>selectedCandidate</code> follow-up.
GET /api/v1/resolve	Unauthenticated scripting endpoint for one-shot track resolves. Supports URLs, free text, and structured search. Ambiguous searches return 400 because the follow-up state is not carried. Genre discovery is not supported here.

3 Query Shapes

3.1 Streaming-Service URL

Use this shape when you already have a track, album, or artist link.

```
https://open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L
https://music.apple.com/de/album/example/123456789
open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L
https://link.deezer.com/s/abc123
```

The resolver detects the service, reads the source metadata, and searches for matching links on other platforms. Schemeless host forms (without `https://`) are accepted for allow-listed music hosts. Short links (for example `link.deezer.com`) are expanded first. Tracking parameters are removed before persistence and before the response is returned.

A request and its (trimmed) response look like this. The `links` array carries one entry per platform the track was matched on, each with a confidence and the `matchMethod` that produced it (`isrc`, `upc`, `search`, `cache`, or `isrc-inference`):

```
GET /api/v1/resolve
?query=https://open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L

200 OK
{
  "id": "I74PsoH_M-oDufeUQW3WT",
  "shortUrl": "https://musiccloud.io/THmwY",
  "track": {
    "title": "Take on Me",
    "artists": ["a-ha"],
    "albumName": "Hunting High and Low",
    "artworkUrl": "https://i.scdn.co/image/ab67616d0000b273e8dd...",
    "durationMs": 225280,
    "isrc": "USWB19901214",
    "releaseDate": "1985-06-01",
    "isExplicit": false,
    "previewUrl": "https://cdnt-preview.dzcdn.net/api/1/.../...mp3"
  },
  "links": [
    { "service": "spotify", "displayName": "Spotify",
      "url": "https://open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L",
      "confidence": 1, "matchMethod": "isrc" },
    { "service": "apple-music", "displayName": "Apple Music",
      "url": "https://music.apple.com/us/album/take-on-me/747796769",
      "confidence": 1, "matchMethod": "isrc" }
  ]
}
```

In the web app, the same resolve renders as a result card with the cover, a preview player, a share action, and the cross-service “Listen on” links:



A resolved track with cross-service links and artist info.

On POST, album and artist URLs resolve as well and return type: "album" or type: "artist" instead of a track. The GET endpoint resolves track URLs only.

3.2 Free-Text Query

Free text is any input that is not a URL and does not start with a reserved query prefix.

```
bohemian rhapsody queen
radiohead karma police ok computer
daft punk one more time
```

This shape is useful when the user only roughly knows what they want. The whole string is used as both the title and the artist term for each adapter; there is no client-side split. A high-confidence match resolves directly to a track response. If there are multiple plausible matches, POST /api/v1/resolve returns a candidate list (see below), while GET returns 400 because it cannot carry the follow-up.

```
POST /api/v1/resolve
{ "query": "daft punk one more time" }
```

```
200 OK
{
  "status": "disambiguation",
```

```

"candidates": [
  { "id": "spotify:ODiWol3A06WpXZgpOgoxAV",
    "title": "One More Time", "artists": ["Daft Punk"],
    "albumName": "Discovery",
    "artworkUrl": "https://i.scdn.co/image/ab67616d0000b273..." },
  { "id": "spotify:1hLNqXjr5371Rd31lz5Gof",
    "title": "One More Time (Short Radio Edit)", "artists": ["Daft Punk"],
    "albumName": "One More Time" }
]
}

```

The web app shows that candidate list as a “Did you mean?” panel with up to eight rows per page:



An ambiguous free-text search resolves to a candidate list.

3.3 Genre-Discovery Query

Genre discovery always starts with `genre: .` It is meant for browsing, not for finding one specific track, and is supported on POST only.

```

genre: ?
genre: jazz
genre: jazz, tracks: 20, vibe: mixed
genre: jazz|r&b, count: 5

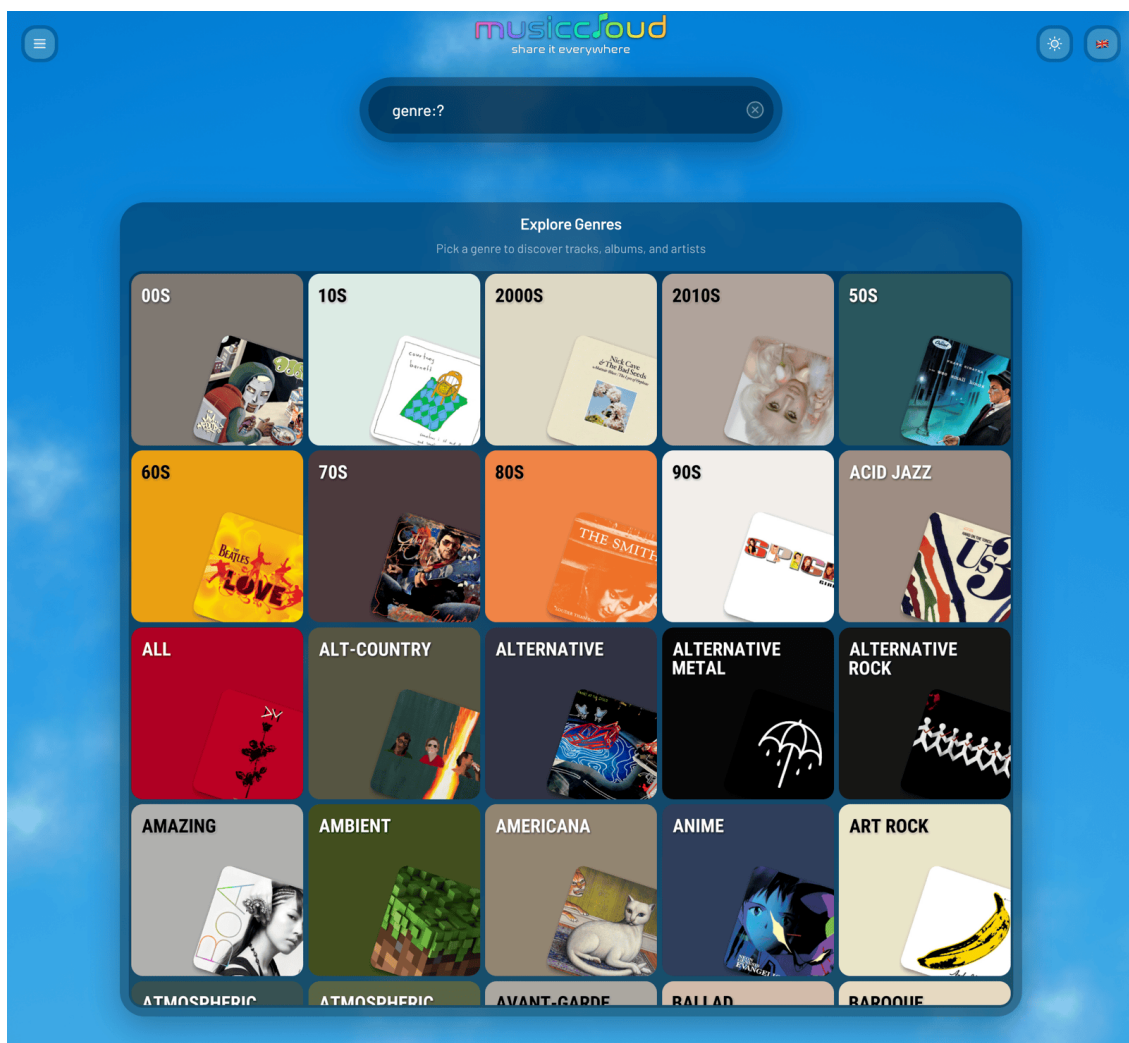
```


genre: ? returns a browse grid of popular genres. Each tile carries a name that you feed back into a genre: <name> query:

```
POST /api/v1/resolve
{ "query": "genre: ?" }
```

200 OK

```
{
  "status": "genre-browse",
  "genres": [
    { "name": "jazz", "displayName": "Jazz",
      "artworkUrl": "/api/genre-artwork/jazz?v=5",
      "accentColor": "#3b2f63" },
    { "name": "synthpop", "displayName": "Synthpop",
      "artworkUrl": "/api/genre-artwork/synthpop?v=5" }
  ]
}
```



The genre: ? browse grid of popular genres.

genre: <name> returns up to three parallel result lists for tracks, albums, and artists. Each list may be null when that type was not requested. The optional modifiers are tracks, albums, artists, count, and vibe (hot or mixed). Track and album rows carry a `webUrl` that you feed into a follow-up resolve to produce a result card.

```
POST /api/v1/resolve
```

```
{ "query": "genre: jazz, tracks: 10, artists: 3" }

200 OK
{
  "status": "genre-search",
  "query": {
    "genres": ["jazz"], "vibe": "hot",
    "tracks": 10, "albums": null, "artists": 3
  },
  "results": {
    "tracks": [
      { "id": "deezer:1109731", "title": "Take Five",
        "artists": ["The Dave Brubeck Quartet"],
        "albumName": "Time Out", "durationMs": 324000,
        "webUrl": "https://www.deezer.com/track/1109731" }
    ],
    "albums": null,
    "artists": [
      { "id": "deezer:1716", "name": "Miles Davis",
        "webUrl": "https://www.deezer.com/artist/1716" }
    ]
  },
  "warnings": []
}
```



A genre: search renders as three result columns.

4 Structured Search

Structured search starts with `title:`, `artist:`, or `album:`. It is meant for concrete track searches where individual fields are known.

4.1 Purpose

Structured search is more precise than free text because title, artist, and optional album can be passed to adapters separately. Services with field operators, such as Spotify, Deezer, or MusicBrainz, can search more narrowly. Other services receive the fields as combined search terms.

4.2 Syntax

```
title: The Killing Moon, artist: Echo & The Bunnymen
artist: Radiohead
title: Karma Police, artist: Radiohead, album: OK Computer, count: 5
```

Field	Meaning
title	Track title. At least one of title or artist is required.
artist	Artist name. At least one of title or artist is required.
album	Optional album title for refinement. album alone is rejected.
count	Optional integer from 1 to 10. Caps the candidate list when the search is ambiguous.

Field names are case-insensitive. Values preserve their original casing. Fields are separated by commas. A missing comma before the next known field is tolerated, for example `title: foo artist: bar`. Commas inside values are not escaped and act as field separators.

4.3 Validation

Empty values, duplicate fields, unknown fields, and `count` outside 1 to 10 return 400. Genre modifiers such as `tracks`, `albums`, `artists`, and `vibe` are intentionally rejected in structured search and belong in `genre: queries`.

4.4 Response

With high confidence, the resolver returns a track response directly, identical in shape to the URL resolve above. With multiple plausible matches, `POST /api/v1/resolve` returns a candidate list. `count` caps this list in addition to the server-side maximum. `GET /api/v1/resolve` cannot continue a candidate selection and returns 400 when the query is ambiguous:

```
GET /api/v1/resolve?query=artist:%20Radiohead
```

```
400 Bad Request
```

```
{
  "error": "MC-URL-0003",
  "message": "Structured query was ambiguous; use POST endpoint
              for disambiguation. (MC-URL-0003)"
}
```


5 selectedCandidate Follow-Up

When a search is ambiguous, POST /api/v1/resolve returns a response with status: "disambiguation" and candidates. Each candidate contains an opaque id. Send that id back unchanged as selectedCandidate, and the resolver returns the final track response:

```
POST /api/v1/resolve
{ "selectedCandidate": "spotify:0DiWol3A06WpXZgp0goxAV" }

200 OK
{
  "id": "Xa3kPq7r-T0mfZ2Lc9D1S",
  "shortUrl": "https://musiccloud.io/9Kdp2",
  "track": {
    "title": "One More Time", "artists": ["Daft Punk"],
    "albumName": "Discovery", "isrc": "GBDUW0000059"
  },
  "links": [
    { "service": "spotify", "displayName": "Spotify",
      "url": "https://open.spotify.com/track/0DiWol3A06WpXZgp0goxAV",
      "confidence": 1, "matchMethod": "isrc" }
  ]
}
```

Clients should not construct the ID themselves. The format is internal and may vary by service.

6 Error Cases

Errors carry a stable, machine-readable error code and a human-readable message that ends with the same code in parentheses:

```
404 Not Found
{
  "error": "MC-RES-0001",
  "message": "This track doesn't seem to be available anymore
             on the source service. (MC-RES-0001)"
}
```

Status	Typical Cause
400	Malformed query, unknown field, empty structured-search values, ambiguous GET search, or invalid selectedCandidate.
401	Missing or invalid credentials on protected endpoints.
404	The query is valid, but no matching track, album, or artist was found.
408	An upstream service did not answer in time.
429	Rate limit exceeded.
503	Required upstream service is temporarily unavailable.

7 Practical Examples

```
# Resolve a Spotify track URL (one-shot, unauthenticated)
GET /api/v1/resolve
?query=https://open.spotify.com/track/2Wfa0iMkCvy7F5fcp2zZ8L
```

```
# Structured search, capped to 3 candidates
POST /api/v1/resolve
{ "query": "title: Karma Police, artist: Radiohead, count: 3" }

# Genre discovery: 10 tracks and 5 albums of jazz
POST /api/v1/resolve
{ "query": "genre: jazz, tracks: 10, albums: 5" }

# Continue a disambiguation by selecting a candidate
POST /api/v1/resolve
{ "selectedCandidate": "deezer:3135556" }

# Free text via the scripting endpoint (URL-encoded)
GET /api/v1/resolve
?query=title:%20Bohemian%20Rhapsody,%20artist:%20Queen
```